

```
# GradCafe Admissions Analyzer
```

Fresh Install Instructions

To run this project from scratch on a new machine, you must install the required dependencies and perform an editable install of the package. You can do this using standard ``pip`` or the faster ``uv`` package manager.

Option 1: Standard Installation (pip)

Activate your virtual environment, then run the following commands to install the frozen dependencies and install the local package in editable mode:

```
```bash
```

```
pip install -r requirements.txt
```

```
pip install -e .
```

## Step 3: Database Hardening (Least Privilege Required)

**Permissions Granted:** I created a dedicated, least-privilege database user named `gradcafe_web`. I granted this user `CONNECT` on the database, `USAGE` on the public schema, and strictly `SELECT` permissions on the `applicants` table. **Why:** By adhering to the Principle of Least Privilege, the web application can only read data. If the application is ever compromised, the attacker cannot `DROP` the table, `ALTER` the schema, or `INSERT/UPDATE` malicious data, because the `gradcafe_web` user fundamentally lacks those administrative privileges.

```
localhost | postgres
1 CREATE USER gradcafe_web WITH PASSWORD 'jhu_secure_pass_2026';
2 GRANT CONNECT ON DATABASE postgres TO gradcafe_web;
3 GRANT USAGE ON SCHEMA public TO gradcafe_web;
4 GRANT SELECT ON TABLE applicants TO gradcafe_web;|
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS POSTGRESQL QUERY RESULTS

**Messages**

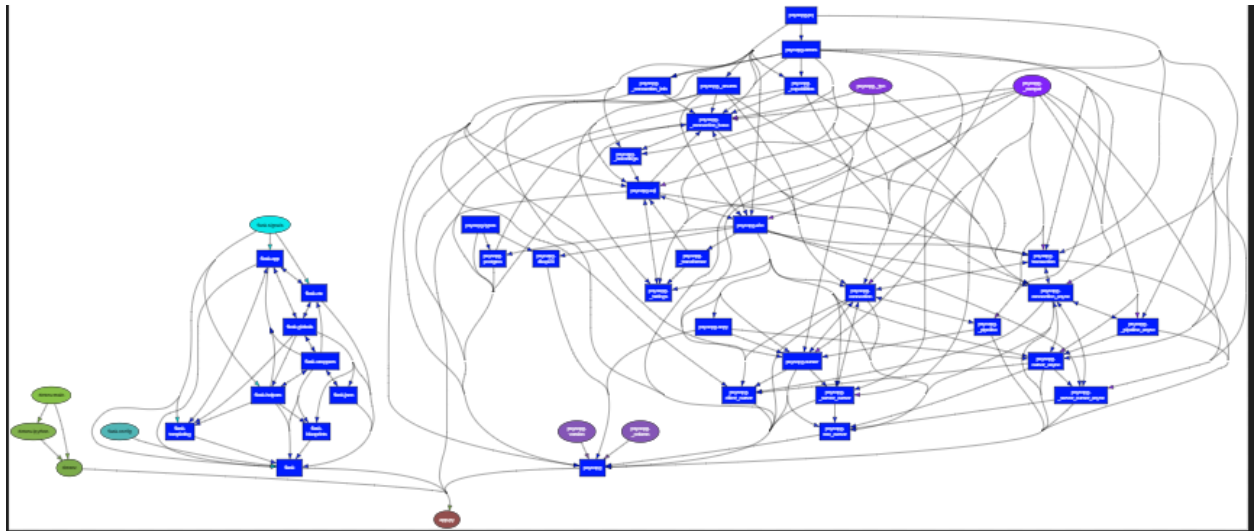
```
12:31:22 PM Started executing query at Line 1
CREATE ROLE
12:31:22 PM Started executing query at Line 2
GRANT
12:31:22 PM Started executing query at Line 3
GRANT
12:31:22 PM Started executing query at Line 4
GRANT
Total execution time: 00:00:00.059
```

## Step 4: Python Dependency Graph (pydeps + Graphviz)

In **5–7 sentences**, explain the key dependencies shown and what they do.

The dependency graph for `app.py` illustrates the interconnected architecture of our admissions data web application. At the core is the **Flask** framework, which handles the web routing, HTTP requests, and rendering of the HTML user interface. For database operations, the application heavily relies on **psycopg** and its **sql** module to establish secure PostgreSQL connections and compose parameterized, injection-safe database queries. The built-in **threading** module is imported to allow the web server to trigger our custom scraper asynchronously in the

background without freezing the webpage. Additionally, the **os** and **dotenv** modules are utilized to securely load environment variables, ensuring that sensitive database credentials are never hardcoded into the source code. Finally, the application integrates our custom **src.scrapy** module, which executes the actual BeautifulSoup data extraction process from TheGradCafe when requested by the user.



### why packaging matters?

Adding a `setup.py` file makes the project an officially installable Python package. It allows us to perform an "editable install" (`pip install -e .`), which forces Python to recognize our local `src/` directory as a proper module. This permanently solves the notorious "it works on my machine" import path issues, ensuring that statements like `from src.scrapy import run_scraper` execute consistently across local runs, testing environments, and CI/CD pipelines.

## Step 6: Snyk Dependency Scan (Required) + Extra Credit

I installed the Snyk CLI and ran the `snyk test` command against my `requirements.txt` file to check for open-source supply chain vulnerabilities. The scan tested all 26 dependencies and

found 0 known issues or vulnerable paths. Because no vulnerabilities were detected, no package patching or removal was required. (See attached `snyk-analysis.png`).

## Extra Credit (+5 points): Snyk Code (SAST)

Run:

```
snyk code test
```

```
Testing C:\Users\Masaomi Enami\Python Project\jhu_software_concepts\Module_5 ...

Open Issues

X [MEDIUM] Use of Hardcoded Passwords
Finding ID: c92c826e-194f-4b7b-9c1e-75297e1a4b44
Path: src/load_data.py, line 17
Info: Do not hardcode passwords in code. Found hardcoded password used in a dictionary key.

X [MEDIUM] Debug Mode Enabled
Finding ID: b5669d53-5056-4ba6-891b-ed73140dc3f
Path: src/app.py, line 278
Info: Running the application in debug mode (debug flag is set to True in run) is a security risk if the application is accessible by untrusted parties.

Test Summary
Organization: masaomienami1
Test type: Static code analysis
Project path: C:\Users\Masaomi Enami\Python Project\jhu_software_concepts\Module_5

Total issues: 2
Ignored issues: 0 [0 HIGH 0 MEDIUM 0 LOW]
Open issues: 2 [0 HIGH 2 MEDIUM 0 LOW]

Tip
To view ignored issues, use the --include-ignores option.
```